



**Università degli Studi di Padova**

Laurea: Informatica

Corso: Ingegneria del Software

Anno Accademico: 2025/2026



**Gruppo RubberDuck**

email: [GroupRubberDuck@gmail.com](mailto:GroupRubberDuck@gmail.com)

**Verbale riunione**  
**2026-05-04**

|                     |                           |
|---------------------|---------------------------|
| <b>Stato</b>        | Approvato                 |
| <b>Versione</b>     | 1.0.0                     |
| <b>Autori</b>       | Felician Mario Necsulescu |
| <b>Verificatori</b> | Ana Maria Draghici        |
| <b>Uso</b>          | Interno                   |
| <b>Destinatari</b>  | Tutto il gruppo           |

## Indice

|                                     |   |
|-------------------------------------|---|
| 1) Informazioni generali .....      | 1 |
| 2) Ordine del giorno .....          | 2 |
| 3) Riassunto della Riunione .....   | 3 |
| 4) Specifica Tecnica .....          | 4 |
| 5) Gestione Asset ed Evidence ..... | 4 |
| 6) Evaluation Engine .....          | 4 |
| 7) Convenzioni Command .....        | 4 |
| 8) Frontend .....                   | 5 |
| 9) Retrospettiva Sprint .....       | 5 |
| 10) Decisioni .....                 | 6 |
| 11) TODO .....                      | 7 |

## 1) Informazioni generali

- **Tipo di riunione:** Interno
- **Motivazione:** Riunione di fine sprint
- **Data:** 2026-05-04
- **Luogo:** Riunione su Discord
- **Ora inizio:** 15.00
- **Ora fine:** 16.00
- **Scriba:** Felician Mario Necsulescu
- **Partecipanti:**
  - Aldo Bettega
  - Davide Testolin
  - Felician Mario Necsulescu
  - Filippo Guerra
  - Ana Maria Draghici
  - Davide Lorenzon

## 2) Ordine del giorno

- Revisione approfondita del dominio e dei diagrammi UML
- Chiarimento delle responsabilità delle classi principali
- Definizione delle convenzioni progettuali (command e service)
- Retrospettiva dello sprint

### 3) Riassunto della Riunione

La riunione è stata dedicata principalmente alla revisione del modello di dominio in vista dell'avvio della fase di codifica. Il gruppo ha analizzato nel dettaglio i diagrammi delle classi, discutendo le responsabilità delle principali entità e il flusso di valutazione dei requisiti.

Sono state prese decisioni rilevanti riguardo alla struttura dei service, all'utilizzo dei command e alla gestione degli output della valutazione. Inoltre, sono stati identificati alcuni aggiornamenti necessari alla Specifica Tecnica.

La seconda parte della riunione è stata dedicata alla retrospettiva dello sprint, con rendicontazione delle attività svolte e riallocazione delle ore tra progettazione e codifica.

## 4) Specifica Tecnica

È stata effettuata una revisione approfondita del dominio, con analisi delle principali entità e delle loro responsabilità.

Il Compliance Standard è stato confermato come contenitore di una collezione di requisiti. Ogni Requirement è caratterizzato da:

- informazioni anagrafiche,
- una lista di dipendenze rappresentate tramite identificativi,
- un decision tree responsabile della logica di valutazione.

Il metodo evaluate del requisito riceve in input:

- un dizionario in sola lettura contenente le risposte,
- i risultati delle dipendenze già calcolati.

Sono stati definiti tre metodi di valutazione:

- evaluateDevice,
- evaluateAsset,
- evaluateRequirement (privato).

È stato chiarito che il metodo principale da utilizzare è evaluateAsset, in quanto gestisce automaticamente la valutazione delle dipendenze tramite memorizzazione dei risultati.

## 5) Gestione Asset ed Evidence

Le evidenze rappresentano proprietà dell'asset e descrivono le caratteristiche utilizzate nel processo di valutazione.

Gli oggetti asset evidence:

- sono mutabili,
- contengono risposta e giustificazione,
- sono gestiti separatamente dall'asset.

Questa scelta è stata motivata dalla volontà di rispettare il principio di singola responsabilità e migliorare la manutenibilità del codice.

## 6) Evaluation Engine

L'Evaluation Engine è stato definito come componente completamente stateless, privo di attributi interni.

Esso espone i metodi di valutazione e delega la logica alle entità del dominio. Include inoltre un metodo di aggregazione che combina i risultati: se almeno un requisito fallisce, l'intero stato aggregato risulta fallito.

## 7) Convenzioni Command

È stato deciso di standardizzare l'utilizzo dei command nei service.

Tutti i service dovranno ricevere un command come parametro, anche nei casi in cui sia necessario un solo dato in input.

I command verranno implementati tramite Pydantic per:

- semplificare la deserializzazione,
- migliorare la validazione dei dati,
- rendere più uniforme la comunicazione tra i livelli applicativi.

## 8) Frontend

È stato discusso il design del frontend, in particolare l'introduzione di elementi aggiuntivi come indicatori di avanzamento. Il gruppo ha concordato di implementare inizialmente solo le funzionalità essenziali e rimandare eventuali miglioramenti estetici a una fase successiva.

È stato inoltre sottolineato che elementi dinamici complessi potrebbero aumentare significativamente la complessità di integrazione con lo stato applicativo.

## 9) Retrospettiva Sprint

Durante la retrospettiva è emerso che una parte significativa del lavoro è stata dedicata alla progettazione del dominio e alla definizione dell'architettura. Alcune attività inizialmente considerate come codifica sono state riclassificate come attività di progettazione.

È stato inoltre evidenziato che i prossimi sprint saranno limitati nel tempo e che sarà necessario ottimizzare l'allocazione delle ore residue.



## 10) Decisioni

| Codice  | Descrizione                                     | Motivazioni                                                | Ref.                    |
|---------|-------------------------------------------------|------------------------------------------------------------|-------------------------|
| VI.26.1 | Standardizzare l'uso dei comand nei service     | Uniformare le interfacce e migliorare la gestione dei dati | <u><b>Sezione 4</b></u> |
| VI.26.2 | Utilizzare evaluateAsset come metodo principale | Gestire correttamente le dipendenze tra requisiti          | <u><b>Sezione 4</b></u> |
| VI.26.3 | Separare asset ed evidenze                      | Applicare il principio di singola responsabilità           | <u><b>Sezione 4</b></u> |
| VI.26.4 | Prioritizzare le funzionalità core del frontend | Ridurre complessità e tempi di sviluppo                    | <u><b>Sezione 9</b></u> |

## 11) TODO

I TODO sorti da questa riunione sono i seguenti:

| Codice  | Assegnatari                                                                          | Task                                                                                 | Decisione di riferimento  |
|---------|--------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|---------------------------|
| TD.31.1 | Ana Maria Draghici                                                                   | Aggiornare i diagrammi del dominio e continuare la redazione della Specifica Tecnica | VI.26.1, VI.26.2, VI.26.3 |
| TD.31.2 | Davide Lorenzon,<br>Felician Mario Necsulescu,<br>Davide Testolin,<br>Filippo Guerra | Continuare la codifica                                                               | -                         |